

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«СЕВЕРО-КАВКАЗКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ ПЕРСПЕКТИВНОЙ ИНЖЕНЕРИИ
ДЕПАРТАМЕНТ ЦИФРОВЫХ, РОБОТОТЕХНИЧЕСКИХ СИСТЕМ
И ЭЛЕКТРОНИКИ**

КУРСОВАЯ РАБОТА

по дисциплине «Интернет-технологии»

Тема: «Разработка веб-приложения для планирования поездок»

Выполнил:

Кравченко Владислав Витальевич

Студент 4 курса группы: ПИН-Б-3-2022

Направления подготовки

09.03.03 Прикладная информатика

Заочной формы обучения

Проверил:
Старший преподаватель департамента цифровых, робототехнических систем
и электроники института перспективной инженерии
Щеголев А. А.

2026

ЗАДАНИЕ

1. Тема курсовой работы: «Разработка веб-приложения для планирования поездок».
2. Цель работы: разработать полноценное веб-приложение для планирования туристических поездок, включающее клиентскую и серверную части, систему аутентификации, инструменты для составления маршрута, учёта расходов и формирования списка вещей.
3. Технологический стек: Python, Django, Django REST Framework, React, TypeScript, Docker.
4. Требования к результату: работающее веб-приложение, развёрнутое в Docker-контейнерах, с документацией и инструкцией по развёртыванию.

СОДЕРЖАНИЕ

1. Введение
2. Проектирование приложения
3. Технологический стек
4. Реализация серверной части
5. Реализация клиентской части

6. Тестирование

7. Заключение

Список использованных источников

Введение

В современном мире путешествия стали неотъемлемой частью жизни. Для эффективной организации поездок необходимо учитывать множество факторов: маршрут, расходы, даты, список необходимых вещей. Ручное планирование часто приводит к ошибкам и недопониманию. Веб-приложение для планирования поездок позволяет систематизировать весь процесс подготовки к путешествию.

Проектирование приложения

Приложение построено по клиент-серверной архитектуре. Серверная часть реализована на Django с использованием Django REST Framework и предоставляет API для работы с данными. Клиентская часть разработана на React с использованием TypeScript и взаимодействует с сервером по REST API.

Основные сущности приложения: пользователь, поездка, пункт маршрута, расход, вещь в чемодане, бронирование и настройки пользователя.

Технологический стек

- Backend: Python 3.10, Django 5, Django REST Framework, JWT-аутентификация
- Frontend: React 18, TypeScript, Vite, Tailwind CSS
- База данных: SQLite (для разработки)
- Контейнеризация: Docker, Docker Compose, Nginx
- Контроль версий: Git, GitHub

Реализация серверной части

Серверная часть включает приложения accounts (аутентификация и настройки пользователя) и trips (управление поездками, маршрутом, расходами, бронированиями). API построено на основе ViewSet'ов Django REST Framework. Аутентификация выполняется с помощью JWT-токенов.

Реализация клиентской части

Клиентская часть представляет собой одностраничное приложение. Основные страницы: авторизация и регистрация, список поездок, создание/редактирование поездки, детальная страница поездки с вкладками маршрута, расходов, чемодана, бронирований, карты и рекомендаций, а также страница настроек.

Реализована поддержка тёмной и светлой темы, переключение языка (русский/английский), выбор валюты, единиц расстояния и температуры.

Тестирование

Проведено функциональное тестирование основных сценариев: регистрация и вход пользователя, создание поездки, добавление маршрута, расходов и вещей, проверка бюджета, развёртывание в Docker. Приложение развёрнуто на удалённом сервере и доступно по IP-адресу.

Заключение

В ходе выполнения курсовой работы было разработано полноценное веб-приложение для планирования поездок. Приложение имеет клиентскую и серверную части, систему аутентификации, возможность развёртывания в Docker. Полученный результат соответствует поставленным требованиям.

Список использованных источников

1. Django Documentation. – <https://docs.djangoproject.com/>
2. Django REST Framework Documentation. – <https://www.django-rest-framework.org/>
3. React Documentation. – <https://react.dev/>
4. TypeScript Documentation. – <https://www.typescriptlang.org/>
5. Tailwind CSS Documentation. – <https://tailwindcss.com/>
6. Docker Documentation. – <https://docs.docker.com/>